📘 NLP Assignment: Progressive Text Classification Techniques

Objective

This assignment aims to develop a deep understanding of text classification by progressively implementing models from basic to advanced levels. Students will explore various neural network architectures, embedding techniques, and evaluation methods to classify text data effectively.

📂 Dataset Selection

Dataset: 20 Newsgroups Text Classification DatasetKaggle+2Kaggle+2Kaggle+2

- **Description:** A collection of approximately 20,000 newsgroup documents, partitioned across 20 different categories. This dataset is widely used for benchmarking text classification algorithms.

🛠️ Assignment Tasks

1. Data Preparation

- **Load the Dataset:**
  - Import the dataset into your working environment.
- **Preprocessing:**
  - Clean the text data by removing noise (e.g., punctuation, stopwords).
  - Tokenize the text.
  - Convert text to lowercase.
- **Data Splitting:**
  - Divide the dataset into training (70%), validation (15%), and testing (15%) sets.
  - Ensure that the same split is used consistently across all models to maintain fair comparisons.

2. Baseline Model: Artificial Neural Network (ANN)

- **Implementation:**
  - Build a simple ANN model for text classification.
  - Use basic text vectorization techniques (e.g., Bag of Words or TF-IDF).
- **Evaluation:**
  - Compute and display the confusion matrix.
  - Calculate accuracy, precision, recall, and F1-score.arXiv+8Kaggle+8Kaggle+8

3. Convolutional Neural Network (1D CNN)

- **Implementation:**
    - Develop a 1D CNN model to capture local features in text sequences.
    - Experiment with different kernel sizes and filter numbers.
- **Evaluation:**
    - Compute and display the confusion matrix.
    - Calculate accuracy, precision, recall, and F1-score.

4. Recurrent Neural Networks (RNNs)

- **Models to Implement:**
    - Simple RNN
    - Gated Recurrent Unit (GRU)
    - Long Short-Term Memory (LSTM)
    - Bidirectional LSTM (BiLSTM)
    - Bidirectional GRU (BiGRU)Wikipedia
- **N-gram Features:**
    - Extract n-gram features with n=5, 10, and 100.
    - Integrate these features into the RNN models.
- **Evaluation:**
    - For each model and n-gram combination, compute and display the confusion matrix.
    - Calculate accuracy, precision, recall, and F1-score.

5. Incorporating Pre-trained Embeddings

- **Embeddings to Use:**
    - FastText
    - Word2Vec
    - BERTKaggle
- **Procedure:**
    - Integrate each embedding into the models listed in Tasks 2-4.
    - Ensure proper tokenization and embedding alignment.
- **Evaluation:**
    - Compute and display the confusion matrix for each model-embedding pair.
    - Calculate accuracy, precision, recall, and F1-score.

6. Transformer-based Model

- **Implementation:**
    - Develop a simple Transformer architecture suitable for text classification.
    - Alternatively, fine-tune a pre-trained Transformer model (e.g., BERT) on the dataset.
- **Evaluation:**
    - Compute and display the confusion matrix.
    - Calculate accuracy, precision, recall, and F1-score.

7. Large Language Models (LLMs) Evaluation

- **Models to Use:**
    - Select pre-trained LLMs within the 100M to 2B parameter range (e.g., DistilBERT, RoBERTa-base,flan-t5, phi1,phi2,deepseekr1-1.5,gemma 1b).
- **Evaluation Scenarios:**
    - **Zero-shot Classification:**
        - Without Example, evaluate the model's ability to classify texts.
    - **One-shot Classification:**
        - Do the Inference model with one example per class and evaluate.
    - **Few-shot Classification:**
        - Do Inference the model with a small number of examples per class (e.g., 5-10) and evaluate.
- **Evaluation:**
    - For each scenario, compute and display the confusion matrix.
    - Calculate accuracy, precision, recall, and F1-score.

8. Results Analysis

- **Comparative Analysis:**
    - Compare the performance metrics across all models and scenarios.
    - Discuss the impact of different embeddings and architectures on model performance.
    - Analyze the effectiveness of zero-shot, one-shot, and few-shot learning using LLMs.
- **Visualization:**
    - Include plots such as accuracy vs. model type, confusion matrices, and any other relevant visualizations to support your analysis.